YouTube Integration

Contents

YouTube Integration **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

YouTube Integration

Scope

Using YouTube as the catalogue and playback provider. Rationale: `brain/decisions/ADR-0003-music-provider-youtube.m`

Official requirements

- Subject III: *"The SDK you choose must not do your work for you."* Voting, licences, visibility, delegation and collaborative editing are our logic. YouTube supplies media only.
- YouTube Data API v3 terms and quota rules; YouTube API Services Terms of Service.
- Quota is enforced per project per day in **units**. `search.list` is roughly two orders of magnitude more expensive than a `videos.list` by id.

Project policy

Boundary

- All YouTube calls go through one adapter in `Infrastructure`, behind an interface declared in `Application`. No handler, controller or hub calls YouTube directly.
- The domain stores a `TrackRef` only: video id, title, channel, duration, thumbnail URL. We never store, cache, proxy or re-host audio or video content.
- If YouTube is unavailable the feature degrades: queue and votes still work on cached `TrackRef` data. A provider outage must not take down the event.

Quota discipline

- Prefer `videos.list` by id over `search.list`. Search only when the user is actually typing a new query.
- Debounce client-side search input; do not issue a request per keystroke.
- **Two cache levels** (`brain/decisions/ADR-0023-catalog-cache.md`): `IMemoryCache` in process for hot lookups, and the `track_refs` table as the persistent cache. Lookup order is L1, L2, then YouTube; a provider response populates both. An in-memory-only cache burns the quota again after every restart, and restarts cluster right before a demo.
- Cache the normalised `TrackRef`, not the raw provider payload. Cache the search query to id-list mapping too, so a repeated search does not re-issue `search.list`.

- Entries refresh by age. Nothing is invalidated by hand — track metadata barely changes, and a stale thumbnail is not worth the complexity.
- Log quota-relevant call counts so the daily budget is measurable before it is exhausted.
- Handle `403 quotaExceeded` as a first-class state with a clear client message, not a 500.

Keys and client split

- The Data API key is **server-side only**, from `.env`. It is never present in the mobile bundle.
- Playback runs on the client through the YouTube player SDK. The backend decides *what* and *when*; it never streams media.
- Metadata lookups go through our API so the key stays server-side and results stay cached.

Resilience

- Timeouts on every outbound call, plus retry with exponential backoff on `5xx` and `429` only.
- A circuit breaker around the adapter so a provider outage does not exhaust our thread pool.
- Never retry a `4xx` other than `429`.

Legal and defence notes

YouTube's terms restrict downloading and certain background-playback behaviours. State this as a product constraint at defence rather than discovering it during evaluation.

Checklist

- ☐ All provider access behind one `Infrastructure` adapter
- ☐ Only `TrackRef` metadata persisted, no media
- ☐ Search debounced and cached; `videos.list` preferred over `search.list`
- ☐ Both cache levels present; a restart does not re-burn quota
- ☐ API key server-side only, absent from the mobile bundle
- ☐ `quotaExceeded` handled explicitly
- ☐ Timeout, backoff and circuit breaker configured

References

- YouTube Data API v3 — https://developers.google.com/youtube/v3/docs
- Quota and cost calculator — https://developers.google.com/youtube/v3/determine_quota_cost
- YouTube IFrame Player API — https://developers.google.com/youtube/iframe_api_reference
- YouTube API Services Terms of Service — https://developers.google.com/youtube/terms/api-services-terms-of-service

Related

`../../10-backend/clean-architecture-cqrs/STANDARD.md` · `../../40-security/secrets-management/STANDARD.md`